CADFormer

Decomposition-Based Generation of Executable CAD Programs from Natural Language

Aayush Iyengar · ECE 595 — NLP · Purdue University

ACL 2026 Format · CadQuery Backend · Azure OpenAI GPT-4

Motivation

The Problem

LLMs are great for general code generation — but CAD is needs a different system to be put together:

Spatial reasoning across 3D coordinate systems

Strict ordering — Boolean operations depend on what's already been created

Type constraints — wrong geometric type = silent failure, not runtime error

Silent correctness failures — code can execute yet produce the wrong shape

The Opportunity

Toolformer	Self-supervised tool learning — but only one tool call at a time
DECINT	Hierarchical decomposition — but no execution feedback between steps
Gorilla	Retrieval-augmented API generation — but single API calls only
CADFormer	Combines all three for multi-step, retrieval-grounded CAD synthesis

Paper 1 — Toolformer (Schick et al., NeurIPS 2023)

Main Takeaway: LLMs can self-learn when and how to use tools (ex. calculator, Wikipedia, Q/A, calendar) with minimal human supervision by using loss-based filtering

Main Takeaways

Self-supervised pipeline — no task-specific labels

6.7B Toolformer beats GPT-3 on arithmetic benchmarks (ASDiv, SVAMP)

Filtering mechanism — only keeps tool calls that lower prediction loss

Tool use only helpful >775M params — validates compute threshold

Limitations of Paper

Single tool call only — CAD requires chained, ordered ops

No linked tool usage — output of one tool never feeds another

Limited calculator — only 4 operations; insufficient for geometric math

CAD relevance — inspired CADFormer's concept of tool-augmented generation

Paper 2 — DECINT (Jhamtani et al., ACL 2023)

Core Idea: Interleave step-by-step decomposition with code generation — each new step sees prior code, not just prior text, preventing CoT drift

Main Takeaways

41% correctness on DeCU vs 34% direct, 25% CoT

Data efficient — generalizes from just 10 complex examples

Code-grounded steps — each step anchored to actual generated code, not NL plan

Ablation studies confirm both decomposition and retrieval are essential

Limitations of Paper

No execution feedback — can't branch on intermediate results

Narrow benchmark — DeCU only has ~200 calendar/email utterances

54% ill-formed — not always constrained to target library

CAD relevance — CADFormer directly adopts interleaved decomposition + code-grounded steps

Paper 3 — Gorilla (Patil et al., NeurIPS 2024)

Core Idea: Retriever-Aware Training (RAT) — fine-tune LLaMA-7B on sometimes-imperfect retrieved docs, so the model learns to weigh retrieval evidence, not blindly copy it

Main Takeaways

Beats GPT-4 on TensorFlowHub (83.79% vs 18.20%) zero-shot

Near-zero hallucination when retrieval is used

AST subtree matching — structural correctness over string matching

RAT robustness — remains effective even when API docs change post-training

Limitations of Paper

Single API call — no chaining or branching on results

ML APIs only — HuggingFace, TorchHub, TFHub only; generality unproven

Retrieval quality dependency — bad retrieval drops below no-retrieval baseline

CAD relevance — CADFormer adopts BM25 retrieval of CadQuery docs per step

CADFormer — Implementation

1	2	3	4	5
Preprocess	**Decompose**	**Retrieve**	**Assemble**	**Validate**
Normalize units Resolve implied dims	DECINT-style NL step generation (max 12)	BM25 top-3 CadQuery ops per step	Generate & concat Python fragments	Execute script Collect errors

Elementary Operation Library

34 CadQuery operations across 5 categories:

Model Creation· Boolean ops · Transforms
Features · Patterns

Stored as JSON with name, signature, description, example

Dual purpose: BM25 retrieval DB + valid operation reference set

CAD-Specific Design Choices

Named step references — s1, s2… carry geometry across steps

Context type summary — tracks variable→geometric type (solid, face, edge, wire)

<DONE> token — halts decomposition when complete, max 12 steps

Benchmark Design — CADBench

L1 — Simple

12 prompts

Example:

"Create a cylinder with 10mm radius and 25mm height"

Single CadQuery operation. Tests primitive generation and argument accuracy.

L2 — Multi-Step

12 prompts

Example:

"Create a hollow rectangular tube with 2mm wall thickness and chamfered inlet"

Boolean ops + transforms. Requires ordering and shape dependency tracking.

L3 — Complex

12 prompts

Example:

"Parametric assembly with 6–10 nested constraints, patterns, conditional geometry"

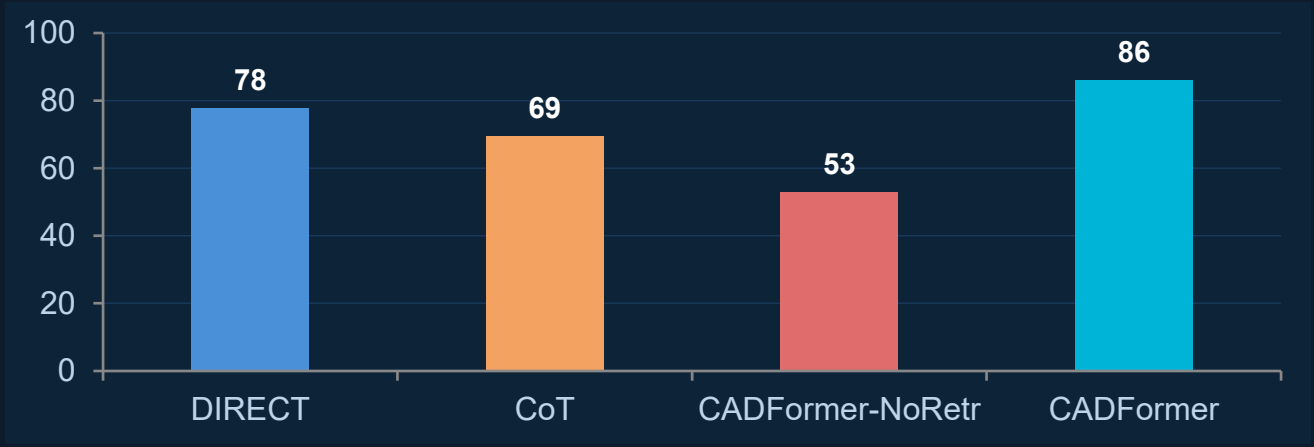
Realistic CAD patterns. Tests full spatial and compositional reasoning.

ESR Execution Success Rate — does the assembled script run without error? **SA** Step Accuracy — valid syntax + only defined variables/operations used?

Results

86.1%

CADFormer ESR (best overall)



▲ **Retrieval is critical —** +33.3 pp gap between CADFormer (86.1%) vs NoRetr (52.8%) — exceeds Gorilla's 12–23% improvement range. CadQuery selector syntax is harder to recall than ML API names.

▼ **CoT hurts vs Direct —** 69.4% vs 77.8% — full NL planning before code introduces format errors without the grounding benefits of code-anchored steps. Unique finding in CAD domain.

△ **L3 > L2 unexpected —** Artifact of small sample (n=12/tier). One failure = 8.33 pp swing. Tier ordering is noisy; larger benchmark needed for robust difficulty scaling claims.

Example Generation (Geometric Correctness)

Simple Prompt: “Create a cylinder with 10mm radius and 25mm height.”

Generated Query: Step 1 Code: s1 = cq.Workplane('XY').circle(10).extrude(25)

Complex Prompt: “Create a phone stand: 80mm × 60mm × 3mm base, 60mm × 3mm × 50mm back wall, joined at the rear of the base, with a 5mm fillet at the junction.“

Generated Query:

Step 1 NL: Create the base plate as a rectangular box with dimensions 80mm 60mm 3mm.
Step 2 NL: Create the back wall as a rectangular box with dimensions 60mm 3mm 50mm, positioned at the rear edge of the base.
Step 3 NL: Union the base plate and the back wall to form a single solid.
Step 4 NL: Apply a 5mm fillet to the inner edge where the base plate meets the back wall.
Step 1 Code: s1 = cq.Workplane('XY').box(80, 60, 3)
Step 2 Code: s2 = cq.Workplane('XY').box(60, 3, 50).translate((0, -28.5, 26.5))
Step 3 Code: s3 = s1.union(s2)
Step 4 Code: s4 = s3.edges().fillet(5)

Error: No CAD Model Generated (b/c one step was incorrect)

Challenges & Failure Analysis

01 Selector Syntax Hallucination

Model creates CadQuery selector strings that look valid but aren't — e.g., "|Z and >Z'" instead of "faces(>Z).edges()". Analogous to Gorilla's API hallucination. BM25 retrieval reduces but doesn't eliminate this.

Fix: Constrained decoding over CadQuery type grammar

02 Ordering Violations

Steps may be individually correct but sequenced wrong — e.g., filleting an edge that doesn't exist yet because the boolean cut creating it comes later. More severe in CAD than calendar/email domains where ordering is irrelevant.

Fix: Topological ordering constraints in decomposition prompt

03 Output Format Inconsistency

DIRECT and CoT baselines sometimes wrap output in markdown fences (```python...) causing immediate syntax errors. Step-by-step generation is naturally more stable since each step is at most one line of code.

Fix: Step-level generation inherently resolves this

Next Steps & Conclusion

Constrained Decoding

Enforce CadQuery type grammar at token level — eliminate selector hallucinations structurally rather than through prompting

Topological Ordering

Add dependency graph checks between decomposition steps — flag if step N references geometry that step N-1 hasn't produced yet

Geometric Evaluation

STEP file comparison against ground-truth shapes — ESR only checks executability, not correctness. Need visual/geometric metric.

APIBench-style Fine-tuning

Create a CadQuery instruction dataset and fine-tune à la Gorilla — reduce GPT-4 API cost and improve reproducibility

CADFormer proves that **retrieval-augmented interleaved decomposition** is viable for NL-to-CAD synthesis.
86.1% ESR · +33pp from retrieval · CoT hurts without code anchoring · Ordering matters uniquely in CAD